Artificial Intelligence

BS (CS) Spring 2026

Project Documentation

Intelligent Reading Comprehension and Quiz Generation System using Machine Learning & Neural Networks

Intelligent Reading Comprehension and Quiz Generation System using Machine Learning & Neural Networks

1. Introduction

Education technology is rapidly growing, and intelligent systems are being used to automate learning and assessment processes. This project proposes an AI-powered Reading Comprehension and Quiz Generation System using the RACE Dataset. The system automatically generates comprehension questions, predicts correct answers, creates distractor options, evaluates user responses, and provides hints.

1.1 Project Overview

This project challenges students to design and implement an intelligent Reading Comprehension (RC) system built on the RACE dataset - one of the largest and most challenging English reading comprehension benchmarks available. The system integrates two distinct AI pipelines and exposes them through an interactive user interface.

What Students Will Build

Model A — Question & Answer Generator / Verifier (Traditional ML models)
Model B — Distractor & Hint Generator (Traditional ML models)
UI Layer — A polished multi-screen application (Streamlit / PyQt5 / React etc.) wiring both models together

1.2 The RACE Dataset

RACE (ReAding Comprehension from Examinations) was originally published by Lai et al. (2017). It is composed of English reading passages and multiple-choice questions extracted from real Chinese middle-school and high-school English examinations in the age range between 12 to 18, RACE consists of near 28,000 passages and near 100,000 questions generated by human experts (English instructors), and covers a variety of topics which are carefully designed for evaluating the students' ability in understanding and reasoning. In particular, the proportion of questions that requires reasoning is much larger in RACE than that in other benchmark datasets for reading comprehension. Its scale and linguistic diversity make it an excellent benchmark for applied ML project.

```
# Load dataset from Kaggle download
import pandas as pd

train_df = pd.read_csv('data/raw/train.csv')
test_df  = pd.read_csv('data/raw/test.csv')
val_df   = pd.read_csv('data/raw/val.csv')

print(train_df.shape)    # Expected: ~87,866 rows
print(train_df.columns) # id, article, question, A, B, C, D, answer
```

Metric	Value
Total Passages	28,000+
Total Questions	~100,000
Question Types	Multiple-choice (A / B / C / D)
Source	Chinese middle & high school English exams
Language	English
Loading Method	pandas.read_csv()
Split	Train / Dev / Test

1.3 Dataset Column Schema

Each row in the dataset contains the following fields:

Column	Type	Description
`id`	String	Unique identifier for the entry in the original dataset
`article`	String	Full reading passage (paragraph-level text)
`question`	String	Multiple-choice question related to the article
`A`	String	Answer option A
`B`	String	Answer option B
`C`	String	Answer option C
`D`	String	Answer option D
`answer`	String	Correct answer label (A, B, C, or D)

2. Project Objectives

By the end of this project, students will be able to:

1. Understand and preprocess a large-scale ML dataset (RACE).
2. Design and train classical ML models for text generation and classification tasks.
3. Implement Traditional ML baselines (Logistic Regression, Naive Bayes, SVM, One-Hot Encoding, etc.) and compare them. Note: TF-IDF vectorization is optional; students are encouraged to use One-Hot Encoding as the primary classical ML feature representation technique.
4. Build a functional AI pipeline integrating two specialized ML models.
5. Develop a user-facing application with a clear, smooth interface.
6. Document model performance using standard ML evaluation metrics.
7. Produce clean, reproducible code and a final project report.

Learning Outcomes at a Glance

ML Engineering — feature engineering, model selection, ensembles, hyperparameter tuning
Software Design — modular architecture, API design, UI/UX principles
Evaluation — Accuracy, F1, Exact Match, RMSE, MAE, MSE, R^2 Score, Confusion Matrix, etc.

3. System Architecture

3.1 High-Level Architecture

The system is divided into three independently deployable layers that communicate through well-defined interfaces:

Layer	Responsibility
Data Layer	RACE dataset loading, preprocessing, feature engineering, train/val/test split management
Model Layer	Model A (Q&A Gen/Verifier) and Model B (Distractor/Hint Gen) training, inference, and persistence
UI Layer	User and developer-facing interface: input handling, result display, feedback loop

3.2 Data Flow Diagram

The data flows through the system in the following order:

```
 [RACE Dataset (CSV/Parquet)]
         |
         v
 [Preprocessing Module]
  • Lowercasing & punctuation removal
  • Handcrafted lexical features & One-Hot Encoding
  • One-Hot Encoding (primary feature representation)
  • TF-IDF vectorization (scikit-learn) [Optional]
  • Cosine similarity matrix
  • Handcrafted lexical features
         |
   +-----+-----+
   |           |
   v           v
[Model A]   [Model B]
Answer       Distractor
Verifier     Ranker +
+ Template   Hint Extractor
Generator
   |           |
   +-----+-----+
         |
         v
 [UI — Streamlit / PyQt5 / React]
```

4. Model A — Question & Answer Generator / Verifier

4.1 Problem Definition

Model A takes a reading passage as input and performs two tasks:

- Generation task: produce a meaningful multiple-choice question and identify the correct answer from the passage.
- Verification task: given an article, a question, and a chosen option, predict whether the selected option is correct.

4.2 Supervised & Unsupervised Learning Models

4.2.1 Supervised Learning Models for Answer Verification

Architecture	Best For	Complexity	Suggested Library
Logistic Regression	Answer verification	Medium	scikit-learn
Support Vector Machine (SVM)	Answer verification & ranking	Low	scikit-learn
Naive Bayes	Question type classification	Medium	scikit-learn
Random Forest / XGBoost	Difficulty estimation & ensemble	Low–Medium	scikit-learn

4.2.2 Unsupervised & Semi-Supervised Approaches

In addition to supervised models, students must explore at least one unsupervised or semi-supervised technique for answer verification and clustering:

- K-Means Clustering: group question-answer pairs by One-Hot Encoded feature similarity to discover latent answer patterns without labels. (TF-IDF feature similarity may also be used as an optional alternative.)
- Label Propagation (Semi-Supervised): use a small labeled set to propagate labels to nearby unlabeled samples in feature space.
- Training: fine-tune bert-base-uncased on RACE train split for 3–5 epochs.
- Gaussian Mixture Models (GMM): probabilistic clustering that assigns soft membership to question-answer clusters, useful for identifying question type patterns.
- Evaluation: compare clustering purity, silhouette score, and semi-supervised F1 against fully supervised baselines.

4.2.3 Template-Based Question Generation with ML Ranking

For the generation sub-task, students apply rule-based templates combined with a trained ML ranker to produce and select meaningful questions:

```
Step 1 — Extract candidate sentences from passage using One-Hot Encoded
keyword overlap with the correct answer. (TF-IDF keyword overlap may
optionally be used as an alternative.)
Step 2 — Apply Wh-word templates (Who/What/Where/When/Why) to transform
candidate sentences into questions. Step 3 — Rank generated questions using
a trained SVM or Random Forest classifier scoring fluency and relevance
features.
```

4.3 Traditional ML Component

Students must implement at least two of the following traditional ML models and compare their performance against the neural approach:

Model	Features Used	Task
Logistic Regression	One-Hot Encoding of (article + question + option) [TF-IDF: optional]	Answer verification
Support Vector Machine	One-Hot Encoding + cosine similarity features [TF-IDF: optional]	Answer verification
Naive Bayes	Bag-of-words of question tokens	Question type classification
Random Forest	Handcrafted lexical features	Difficulty estimation
XGBoost	Semantic similarity + length features	Answer verification

4.4 Ensemble Strategy

Students are encouraged to explore ensembling the neural and traditional ML predictions. Suggested strategies:

- Soft voting: average probability outputs from multiple classifiers (e.g., SVM, Logistic Regression, Naive Bayes).
- Hard voting: majority vote across N models.
- Stacking: train a meta-classifier on the outputs of base models.

4.5 Evaluation Metrics for Model A

Metric	Application
Accuracy	Verification sub-task — fraction of correctly labeled options
Macro F1	Verification sub-task — handles class imbalance
Exact Match (EM)	Verification — strict character-level match to gold label

5. Model B — Distractor & Hint Generator

5.1 Problem Definition

Model B takes a reading passage, a question, and the correct answer as input and produces:

- Three plausible distractors (wrong answer options A, B, C or whichever are incorrect) that are semantically related but factually wrong.
- One or more graduated hints that guide the reader toward the correct answer without giving it away directly.

5.2 Why Distractor Generation is Challenging

Good distractors must satisfy several competing constraints simultaneously:

- Plausibility: they must look like legitimate answers to an uninformed reader.
- Incorrectness: they must be definitively wrong with respect to the passage.
- Diversity: the three distractors should not be trivially similar to each other.
- Grammatical consistency: all options must share the same syntactic form.

5.3 Classical ML Models for Distractor & Hint Generation

5.3.1 Supervised Distractor Ranking

Students implement a supervised pipeline using One-Hot Encoding and ML classifiers to rank candidate distractors extracted from the passage. (TF-IDF may optionally be used as an alternative feature representation.) The pipeline steps are:

```
Step 1 — Candidate extraction: retrieve all candidate phrases from the
passage using simple string matching and frequency-based word selection
(classical ML approach, no NLP tools required).
```

```
Step 2 — Feature engineering: for each candidate compute One-Hot Encoding
cosine similarity to the correct answer, character-level match score, and
passage frequency. (TF-IDF cosine similarity may optionally be used.) Step 3
— ML Ranker: train a Logistic Regression or Random Forest classifier to
score each candidate; select top-3 non-answer candidates as distractors.
```

Alternative approach using One-Hot Encoding Vocabulary + Cosine Similarity (TF-IDF is optional):

- Build a One-Hot Encoded vocabulary over the full training corpus. For each answer token, retrieve the top-N most co-occurring terms as synonym candidates.
- Re-rank candidates using cosine similarity scores computed from One-Hot Encoding representations. (TF-IDF cosine similarity scores may be used optionally.)
- Apply a diversity penalty to ensure variation across the three distractors.

5.3.2 Extractive Hint Generation with ML Scoring

Hints should be generated as extractive or abstractive summaries of the most relevant passage sentences. Two strategies:

- Extractive: score each sentence in the article by its relevance to the question (using cosine similarity of sentence embeddings), then surface the top-K sentences as ranked hints.
- ML-scored: train a Logistic Regression on sentence features (keyword overlap, position in passage, sentence length) to score and rank sentences as hints without using any neural network.

5.4 Traditional ML Component

For the distractor selection and ranking sub-task, implement at least one of:

Approach	Description
One-Hot Encoding + Cosine Similarity (TF-IDF: Optional)	Retrieve candidate phrases from the passage and re-rank by cosine similarity to the answer using One-Hot Encoding representations. Low-similarity high-frequency candidates become distractors. (TF-IDF may be used as an optional alternative.)
Word2Vec Nearest Neighbours	Embed the correct answer with pre-trained Word2Vec (Google News). Retrieve the top-N nearest neighbours that are NOT extractable from the passage.
Frequency-Based Substitution	Identify high-frequency content words in the passage (nouns and proper nouns identified via simple frequency counting and One-Hot Encoding). Substitute the correct answer with similarly frequent but semantically distinct terms to create plausible distractors.
Rule-Based Hint Extraction	Apply a bag-of-important-words overlap heuristic: find passage sentences sharing the most content words with the question. Present them in descending relevance order.

5.5 Evaluation Metrics for Model B

Metric	Application
Precision / Recall / F1	Fraction of generated distractors that are plausible but incorrect (Precision); fraction of reference distractors recovered (Recall); harmonic mean (F1)
Confusion Matrix	Human evaluation (1–5 Likert scale): how believable is each distractor?
Accuracy	Fraction of test samples where the top-ranked distractor candidate is not the correct answer (distractor ranker accuracy).
Precision	Fraction of top-K ranked hint sentences that overlap with the gold key sentence; measures hint extraction quality.
R^2 Score	For regression-based hint scorer: R^2 measures how well predicted sentence relevance scores correlate with true relevance labels.

6. User Interface Design

6.1 Platform Options

Students may implement the UI using any of the following frameworks. The choice must be documented and justified in the final report.

Platform	Language	Best For	Difficulty
Streamlit	Python	Rapid ML prototypes	Easy ✓ Recommended
Gradio	Python	Model demos, sharing	Easy
PyQt5	Python	Desktop native app	Medium
React + Flask	JS + Python	Full-stack web app	Hard
Tkinter	Python	Lightweight desktop	Easy
FastAPI + Jinja2	Python	Web, minimal JS	Medium

6.2 Required Screens / Views

Regardless of platform, the UI must provide the following screens:

Screen 1 — Article Input

- A text area where the user can paste or upload a reading passage.

- Option to load a random sample from the RACE dataset for quick testing.
- A 'Submit' button that triggers both Model A and Model B inference.

Screen 2 — Question & Answer Quiz View

- Displays the generated (or RACE original) question.
- Shows four options (A, B, C, D) — one correct and three distractors from Model B.
- User selects an answer and clicks 'Check' — Model A verifier confirms correctness.
- Shows a colour-coded result (green = correct, red = incorrect) with explanation.

Screen 3 — Hint Panel

- A collapsible or tabbed panel showing graduated hints from Model B.
- Hint 1: most general clue; Hint 2: more specific; Hint 3: near-explicit.
- 'Reveal Answer' button only appears after the user has used all hints.

Screen 4 — Developer / Analytics Dashboard

- Model A performance: Accuracy, F1-Score, Precision, Recall, Confusion Matrix on the last N inferences.
- Model B performance: Precision, Recall, F1-Score, and Accuracy for distractor ranking.
- Inference time per request (latency tracking).
- A table or export button for logging session results to CSV.

6.3 UX Requirements

- The application must be usable without reading a manual.
- All error states (empty input, model failure) must display a friendly message.
- Loading indicators must be shown during model inference.
- The interface must be accessible: sufficient colour contrast, readable font sizes.

7. Recommended Technical Stack

7.1 Core Libraries

Category	Library / Tool	Purpose
Classical ML	`scikit-learn`	Supervised, Unsupervised, Semi-Supervised models
Text Features	`Gensim`	Word2Vec, One-Hot Encoding, bag-of-words feature extraction (TF-IDF: optional)
Traditional ML	`scikit-learn`	Logistic Regression, SVM, Naive Bayes, RF
Gradient Boosting	`XGBoost / LightGBM`	Ensemble boosted classifiers
Embeddings	`sentence-transformers`	Semantic similarity for hint ranking
Data Handling	`pandas / numpy`	Dataset loading, feature matrices
Text Vectorization	`scikit-learn One-Hot Encoder (TF-IDF: optional)`	Bag-of-words feature engineering
Evaluation	`scikit-learn metrics`	F1, Precision, Recall, Confusion Matrix, RMSE, R^2 computation
Model Persistence	`joblib / torch.save`	Save and load trained models
UI — Option 1	`Streamlit`	Python-native web app framework
UI — Option 2	`PyQt5`	Desktop GUI framework
UI — Option 3	`React + FastAPI`	Full-stack web application

7.2 Project Folder Structure

```
race_rc_project/
├── data/
│   ├── raw/                  # Original RACE CSV files
│   └── processed/            # Tokenized, feature-engineered data
├── models/
│   ├── model_a/
│   │   ├── neural/           # BERT / T5 checkpoints
│   │   └── traditional/      # Pickled sklearn models
│   └── model_b/
│       ├── neural/           # Distractor / hint model checkpoints
│       └── traditional/      # Pickled sklearn / word2vec models
├── src/
│   ├── preprocessing.py      # Dataset loading & feature engineering
│   ├── model_a_train.py      # Training script for Model A
│   ├── model_b_train.py      # Training script for Model B
```

```
│   ├── inference.py          # Unified inference API
│   └── evaluate.py           # Metric computation
├── ui/
│   ├── app.py                # Streamlit / PyQt5 / FastAPI entry point
│   └── components/           # Reusable UI components
├── notebooks/
│   ├── EDA.ipynb             # Exploratory Data Analysis
│   └── experiments.ipynb     # Experiment tracking
├── tests/
│   └── test_inference.py     # Unit tests
├── requirements.txt
├── README.md
└── report/
    └── final_report.pdf
```

8. Implementation Plan & Milestones

The project is designed for a 2-week sprint. Below is a suggested daily/weekly plan:

Week(s)	Milestone	Deliverable
Week 1, Days 1–2	**Data Exploration**	EDA notebook: distributions, passage lengths, question types, answer balance. Summary statistics table.
Week 1, Days 3–4	**Preprocessing**	preprocessing.py complete. Train/dev/test splits ready. One-Hot Encoding and embedding feature matrices saved. (TF-IDF matrices: optional.)
Week 1, Days 4–5	**Model A — Trad. ML**	LR and SVM classifiers trained on One-Hot Encoding features. Accuracy + F1 reported on dev set. (TF-IDF features: optional.)
Week 2, Days 1–2	**Model A — Unsupervised/Semi-Supervised**	Label Propagation or K-Means clustering implemented. Purity/Silhouette score reported. Comparison table vs. supervised ML.
Week 2, Day 3	**Model A — Ensemble**	Soft-vote or stacking ensemble implemented. Final Model A results on test set.
Week 2, Day 3	**Model B — Trad. ML**	One-Hot Encoding + cosine similarity distractor pipeline working. Word2Vec nearest-neighbour distractors evaluated using Precision, Recall and F1. (TF-IDF pipeline: optional.)
Week 2, Day 4	**Model B**	ML-ranked distractor pipeline complete. Extractive hint scorer implemented and evaluated.
Week 2, Day 4	**UI Development**	All four screens implemented. Models integrated. Error handling complete.
Week 2, Day 5	**Evaluation & Tuning**	Full evaluation on RACE test set. Hyperparameter sweep. Human evaluation form completed.
Week 2, Day 5	**Final Submission**	Final report (PDF). GitHub repo with README. 10-min demo video or live presentation.

9. Grading Rubric

Component	Marks	Key Criteria
EDA & Preprocessing	**10 / 100**	Insightful visualizations; clean, documented pipeline
Model A — Traditional ML	**15 / 100**	≥ 2 models, feature engineering, metric comparison table
Model A — Unsupervised / Semi-Supervised	**20 / 100**	At least one unsupervised or semi-supervised approach implemented and evaluated

Component	Marks	Key Criteria
Model A — Ensemble	**05 / 100**	Demonstrates improvement over individual models
Model B — Distractor Gen.	**15 / 100**	Plausible distractors; Precision, Recall, F1, and Confusion Matrix reported
Model B — Hint Gen.	**10 / 100**	Graduated hints that guide without revealing answer
User Interface	**15 / 100**	All 4 screens present; smooth UX; error handling
Final Report	**05 / 100**	Clear methodology, results, discussion, limitations
Code Quality	**05 / 100**	Readable, documented code; meaningful commit history
TOTAL	**100 / 100**	

10. Constraints & Ethical Considerations

10.1 Technical Constraints

- All models must be trainable on a standard university GPU (e.g., NVIDIA RTX 3060 12 GB or Google Colab T4). Avoid architectures that require >24 GB VRAM.
- Inference for a single article + question must complete in under 10 seconds on the target hardware.
- The full training pipeline (all models) must be reproducible from a single shell command or Jupyter notebook.

10.2 Ethical Considerations

As future AI practitioners, students must reflect on and address the following in their final report:

- Bias: RACE passages come from Chinese school examinations. Consider whether cultural or linguistic bias in the dataset affects model generalization to other populations.
- Accessibility: the UI must be usable by students with visual impairments (sufficient contrast ratios, keyboard navigation).
- Academic integrity: generated questions must not be deployed in real exam settings without human review.

- Model transparency: the UI must make it clear to users which answers are AI-generated and that errors are possible.

11. Deliverables Summary

Submission Checklist

[ ] GitHub repository (shared with instructor) — clean commit history
[ ] requirements.txt with pinned versions
[ ] README.md with setup, training, and run instructions
[ ] EDA notebook (notebooks/EDA.ipynb)
[ ] Trained model checkpoints (notebooks)
[ ] Final report PDF (proper documentation in neat and paper style)
[ ] 10-minute Live demo session
[ ] Human evaluation forms (Rubric form)

11.1 Final Report Structure

1. Abstract (200 words max)
2. Introduction & Motivation
3. Related Work (cite at least 5 papers)
4. Dataset Analysis
5. Model A: Design, Training, Results
6. Model B: Design, Training, Results
7. User Interface Description
8. Evaluation & Discussion
9. Limitations & Future Work
10. Conclusion
11. References

12. Suggested References

Optional: Students can read the following foundational papers and resources before beginning the project:

Dataset

- Lai, G., Xie, Q., Liu, H., Yang, Y., & Hovy, E. (2017). RACE: Large-scale ReAding Comprehension Dataset From Examinations. EMNLP 2017.

Question Generation

- Du, X., Shao, J., & Cardie, C. (2017). Learning to Ask: Neural Question Generation for Reading Comprehension. ACL 2017.
- Zhao, Y., Ni, X., Ding, Y., & Ke, Q. (2018). Paragraph-level Neural Question Generation with Maxout Pointer and Gated Self-attention Networks. EMNLP 2018.

Distractor Generation

- Guo, Q., Zhu, X., Chao, W., Ye, Z. (2016). Generating Distractors for Reading Comprehension Questions from Real Examinations. AAAI 2016.
- Jiang, Y., & Lee, H.-Y. (2017). What Click-Through Data Can Tell Us About Distractor Quality. BEA Workshop 2017.

Pre-trained Models

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL 2019.
- Raffel, C. et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. JMLR 2020.

Evaluation

- Papineni, K. et al. (2002). BLEU: a Method for Automatic Evaluation of Machine Translation. ACL 2002.
- Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. ACL Workshop 2004.